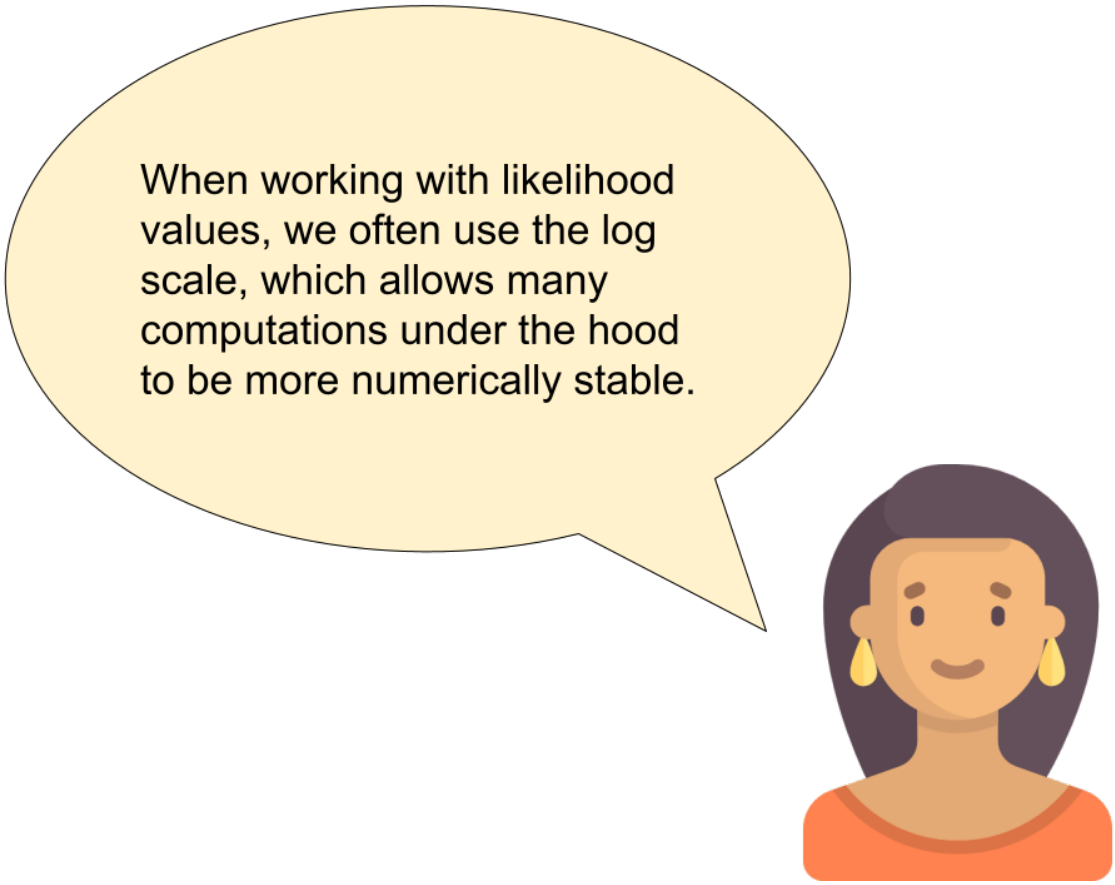


With this log likelihood computation in hand, our last step is to simply find the value for our mean function such that the log likelihood is maximized. In other words, we aim to seek the mean value that explains our training data the best. Since we have access to the log likelihood computation, we can use gradient-based optimization algorithms such as gradient descent to iteratively refine the mean value that we have. Upon convergence, we will have arrived at a good mean value that gives high data likelihood. If you need a refresher on how gradient descent works, I recommend appendix B of Luis Serrano's *Grokking Machine Learning*, which does a good job explaining the concept.

Now, let's see how we can implement this process with code. Since we implemented our GP model with the `gpytorch.means.ConstantMean` class for our mean function, we don't need to change anything here. So for now, let's initialize our GP model again:

```
# declare the GP
lengthscale = 1
noise = 1e-4

likelihood = gpytorch.likelihoods.GaussianLikelihood()
model = ConstantMeanGPModel(train_x, train_y, likelihood)

# fix the hyperparameters
model.covar_module.lengthscale = lengthscale
model.likelihood.noise = noise
```

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>